PROMPTS REPORT

We have used various AI throughout this project in different phases:

1. LLM used - AI QWEN A3 through Ollama	2. Generating backend api : Antigravity
3. Designing ui ux : Stitch + ai studio by google	4. Debug errors and integration : Antigravity
5. Optimizing logic : Claude	

We used a structured prompting approach along with multiple iterations to improve the results step by step. Based on our basic understanding of prompt engineering, we applied different techniques wherever needed. These included the RCTCF method (defining role, context, task, constraints, and format), using step-by-step instructions to guide the AI's reasoning, and using delimiters like quotes or brackets to clearly separate specific inputs or information.

A. Initial problem statement

Prompt 1:

Create a Gen-ai system that can evaluate, provides structured and explainable decision logic for prioritization, architectural choices along with transparency, document trade-offs.

Feedback: too vague, no clear role output or rules are mentioned. AI can think this in generc way which is unnecessary.

Prompt 2:

You are a product strategy AI. Your task is to design a GenAI-powered decision intelligence system. The system must: (1) evaluate and score product options, (2) explain prioritization logic in plain language, (3) document architectural trade-offs clearly. Constraints: avoid technical jargon; output should be usable by non-technical product managers. Format: structured sections with headings — Summary, Problem, Solutions (with scores), Trade-offs, Recommendation.

Feedback: worked on RCT-CF, no extra highlighted constraint

B. Product Strategy Design

Prompt 3:

You are a product strategy consultant specializing in GenAI-driven product decision systems. Design a GenAI-powered Product Decision & Trade-off Intelligence System... Avoid technical jargon, keep features practical, focus on usability. Use clear headings, bullet points, and a logical structure.

Feedback: slightly unclear scoring method and could be explained through an example. Few constraints and features are not stated clearly.

C. FastAPI backend Build

Prompt 4:

Act as a senior backend engineer. Build a complete FastAPI backend (single {main.py}) for a GenAI decision system with a POST /api/analyze endpoint, implement a 5-stage prompt chain using local Ollama (qwen3) instead of external APIs, create a reusable LLM call function, enforce strict JSON parsing and validation with retry on failure, log outputs and timings, and return all stage outputs in a structured response.

D. React Frontend build

Prompt 5:

_Act as a senior frontend engineer. Build a complete React + Tailwind frontend (single App.jsx) for a decision intelligence system with a multi-screen UI (Input, Thinking, Options, Analysis, Recommendation, Audit), modern SaaS dark theme, input form supporting constraints and weighted objectives, options explorer with card layout, trade-off matrix with color-coded scores, radar chart visualization, recommendation dashboard with confidence score, and an audit screen showing reasoning quality and biases; ensure clean, responsive, production-level UI.

E. Integration of backend and frontend

Prompt 6:

_Act as a senior full-stack engineer. Integrate the React frontend with the FastAPI backend by calling POST /api/analyze to obtain a session_id, polling GET /api/status/{session_id} every 1–2 seconds, and updating the UI based on current_stage, progress, and stage outputs, mapping stage2 to the options screen, stage3 to the trade-off matrix, stage4 to the recommendation dashboard, and stage5 to the audit screen.```

Feedback: needs to add timeout handling and error boundary

Prompt 7:

Act as a senior full-stack engineer. Integrate the existing React frontend (App.jsx) with the FastAPI backend (main.py). Integration requirements: (1) On form submit, call POST /api/analyze with the input payload and store the returned session_id, (2) Begin polling GET /api/status/{session_id} every 1.5 seconds using setInterval; stop polling when status is 'complete' or 'error', (3) Map stage responses to screens: stage2 data → Options screen, stage3 data → Analysis/Trade-off screen, stage4 data → Recommendation screen, stage5 data → Audit screen, (4) Handle loading, error, and timeout (>60s) states gracefully with user-visible messages, (5) Use React useState and useEffect only — no external state libraries. Output: updated App.jsx with integration logic inline. Include error boundary for API failures.

The approach thought action observation loop and refine helped a lot increase the efficiency of the prompts and actually made AI think longer.